Netzwerke 1

Abgabe 1

Melvin Lux, Angelina Vorndran

Bachelor Informatik, 3. Semester

Fakultät Informatik und Mathematik

Hochschule München University of Applied Sciences

München, 4. November 2026

1 Blatt 1

Aufgabe 1.1: Netzwerk-Traces mit Wireshark

Teilaufgabe a) Aufruf der Webseite www.cs.hm.edu im Browser

- **Beobachtete Protokolle:** DNS, TCP, UDP, TLSv1.3, QUIC
- **Ablauf & Beschreibung:**
  1. **DNS (Domain Name System):** Löst den Domainnamen `www.cs.hm.edu` in die zugehörige IP-Adresse auf (typischerweise über UDP).
  2. **TCP / UDP:** Transportprotokolle für den Verbindungsaufbau und die Datenübertragung.
  3. **TLSv1.3 / QUIC:** Sichere, verschlüsselte Protokolle zur Übertragung von HTTPS-Webinhalten. QUIC setzt dabei auf UDP auf und kombiniert Verbindungsaufbau sowie Verschlüsselung.

Teilaufgabe b) Ping auf www.google.com

- **Beobachtete Protokolle:** DNS, ICMP / ICMPv6
- **Antwortende IP-Adresse:** `142.251.153.119`
- **Ablauf & Beschreibung:**
  1. **DNS:** Ermittelt die IP-Adresse von `www.google.com`.
  2. **ICMP (Internet Control Message Protocol):** Sendet *Echo Request*-Pakete an die Ziel-IP. Der Rechner antwortet mit *Echo Reply*-Paketen zur Diagnose der Erreichbarkeit und Latenz.

Teilaufgabe c) FTP-Verzeichnisanzeige (ftp.slackware.com)

- **Beobachtete Protokolle:** DNS, TCP, FTP
- **Anmeldedaten (Anonymous-FTP):**
  - Benutzername: `ftp` oder `anonymous`
  - Passwort: z. B. eine E-Mail-Adresse wie `ftp@melvinlux.de`
- **Erzielte Datenrate:** ca. 11,59 MiB/s
- **Sicherheitsproblem von FTP:** FTP überträgt Steuerdaten (einschließlich Benutzername und Passwort im Klartext) sowie alle Nutzdaten unverschlüsselt. Ein Angreifer im selben

Netzwerk kann diese sensiblen Daten problemlos über Wireshark mitlesen.

Einordnung der Protokolle in die Schichten (OSI-Modell)

Protokoll	Schicht im OSI-Modell	Bezeichnung der Schicht
ARP	Schicht 2 / 3	Data Link / Network
ICMP	Schicht 3	Network
TCP / UDP	Schicht 4	Transport
TLSv1.3	Schicht 5	Session
QUIC	Schicht 5 + 6	Session & Presentation
DNS, FTP, NTP	Schicht 7	Application

Tabelle 1: Einordnung der beobachteten Netzwerkprotokolle in das OSI-Referenzmodell.

Aufgabe 1.2: Netzwerkkommunikation ganz ohne Netzwerkprogrammierung

1. Datei von Client zu Server senden

- **Server-Seite:**

```
nc -k -l 4711 > received.txt
```

Erklärung: Der Server lauscht auf Port 4711 (`-l`), bleibt nach Verbindungsende aktiv (`-k`) und leitet alle eingehenden Daten in die Datei `received.txt` um (`>`).

- **Client-Seite:**

```
cat test.txt | nc localhost 4711
```

Erklärung: Der Inhalt der Datei `test.txt` wird über die Pipe (`|`) an den Client `nc` übergeben und an den lokalen Port 4711 gesendet.

2. Netzwerkdienst zur Umwandlung von Texten

- **Server-Seite:**

```
nc -l 4711 < pipe | sed -u 's/Vorlesung/Party/g' > pipe
```

Erklärung:

1. `mkfifo pipe`: Erstellt eine benannte Pipeline (Named Pipe), um eine Rückkopplung/Schleife für Daten zu ermöglichen.
2. `nc -l 4711 < pipe`: Liest die verarbeitete Antwort aus der Named Pipe und schickt sie zurück an den Client.
3. `| sed -u 's/Vorlesung/Party/g'`: Der Stream wird durch `sed` verarbeitet. Jedes Vorkommen von `"Vorlesung"` wird global (`/g`) durch `PParty"` ersetzt. Die Option `-u` (*unbuffered*) stellt sicher, dass der Text sofort ohne Pufferung ausgegeben wird.
4. `> pipe`: Schreibt das Ergebnis zurück in die Named Pipe.

- **Client-Seite:**

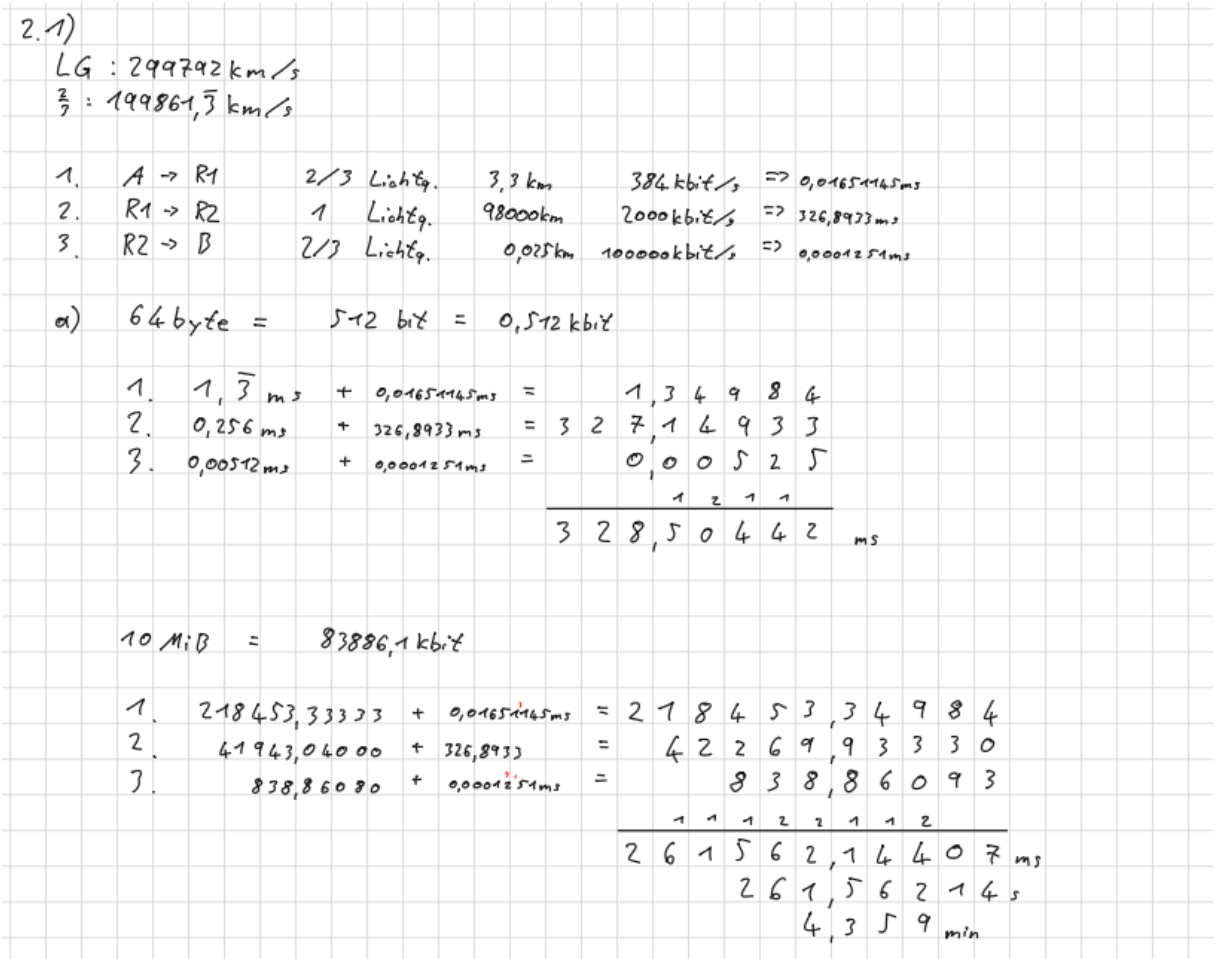
```
nc localhost 4711
```

Erklärung: Baut eine interaktive Verbindung zum Dienst auf Port 4711 auf. Nach der Eingabe von z. B. `Ist morgen Vorlesung?` empfängt der Client die Antwort `Ist morgen Party?`.

2 Blatt 2

2.1 Aufgabe 2.1

2.1.1 a

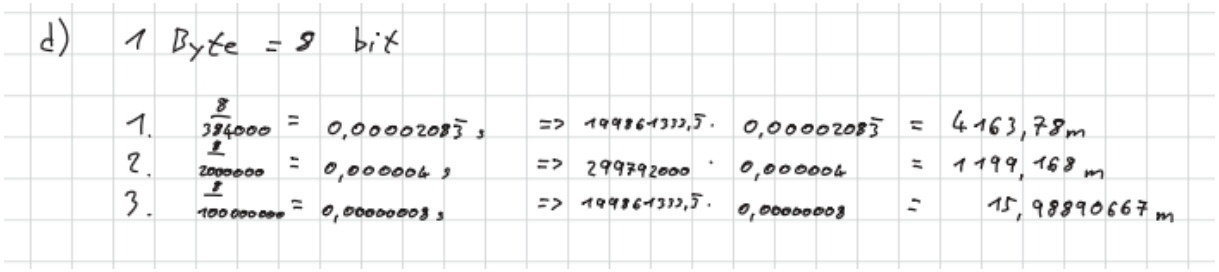


2.1.2 b

2.1.3 c

Wenn man eine der Variablen verändern könnte würden wir die Übertragungsrate vom 1. Schritt (A->R1) erhöhen um die dortigen Zeiten drastisch zu erhöhen.

2.1.4 d



2.1.5 e

2.2 Aufgabe 2.2

2.2.1 a

Das Internet-Schichtenmodell unterscheidet sich vom ISO/OSI-Schichtenmodell hauptsächlich an 2 Stellen, indem es einige Schichten zusammenfasst:

- 1. Verbindungsschicht -> Bitübertragungsschicht & Sicherungsschicht
- 2. Anwendungsschicht -> Kommunikationssteuerung & Darstellungsschicht & Anwendungsschicht

Im Allgemeinen lässt sich sagen, dass das Internet-Schichtenmodell das ISO/OSI-Schichtenmodell simplifiziert und für den Endanwender unnötige Schichten zusammenfasst.

2.2.2 b

Funktionalität wird im Internet immer auf der höchstmöglichen Schicht realisiert, um keine neue Hardware in der Infrastruktur zu benötigen, die Netzneutralität nicht zu gefährden und um weitere Kosten durch den Netzbetreiber vermeiden zu können.

2.2.3 c

Nein. Ein zuverlässiges Schicht-2-Protokoll garantiert die korrekte Übertragung nur zwischen zwei direkt verbundenen Knoten, also pro Teilstrecke. Auf dem Weg zwischen den Endsystemen können Pakete trotzdem verloren gehen (z. B. durch überlaufende Router-Puffer), verfälscht werden (Fehler im Router nach der Link-Prüfung), dupliziert werden oder in falscher Reihenfolge ankommen. Diese Fehler werden auf Schicht 2 nicht erkannt. Zuverlässigkeit muss

daher Ende-zu-Ende gewährleistet werden, also im Transportprotokoll (TCP) oder in der Anwendung. UDP bietet das nicht. Der gesparte Overhead würde die Zuverlässigkeit gefährden.

2.3 Aufgabe 2.3

2.3.1 FreeDiskSpaceServer

Der Vollständige Code ist in FreeDiskSpaceServer.java zu finden.

2.3.2 FreeDiskSpaceClient

Der Vollständige Code ist in FreeDiskSpaceClient.java zu finden.

3 Blatt 3

3.1 3.1

3.1.1 a

$https://cs.hm.edu/aktuelles/news/news_detailseite_133888.de.html$

3.1.2 b

Ja, damit kann umgegangen werden, da im Header Äccept-Encoding"gzip enthalten ist.

3.1.3 c

Es wird eine persistente Verbindung erwartet, da der Header Connection"den Wert keep-alive hat.

3.1.4 d

Wenn der Header Ïf-None-Match"der Server würde ausschließlich Datein zurückgeben die den ETag "ddnenj89j4jf8ejf8juw37EEjh8Suf3nnicht besitzen, dann würde auch der Code 200 gesendet werden, wenn eine solche Datei existiert bekommen wir 304 NNot Modifiedßurück.

3.1.5 e

Der Server kann mit 2 Möglichkeiten antworten:

- 200 — OK

```
❯ curl --compressed "https://cantamus.app/packages/verovio-3.3.0/verovio.js" -H "If-None-Match: f1ee2cc0ca9c51
f2d05575f8c1436407-ssl-df" -I
HTTP/2 200
accept-ranges: bytes
cache-control: public, max-age=0, must-revalidate
content-length: 7442291
content-type: application/javascript
date: Tue, 18 May 2021 14:18:06 GMT
etag: "f1ee2cc0ca9c51f2d05575f8c1436407-ssl"
strict-transport-security: max-age=31536000
age: 4437
x-nf-request-id: 34914077-fc7f-4bf1-82eb-06852a4801d6
server: Netlify
```

- 304 — Not Modified

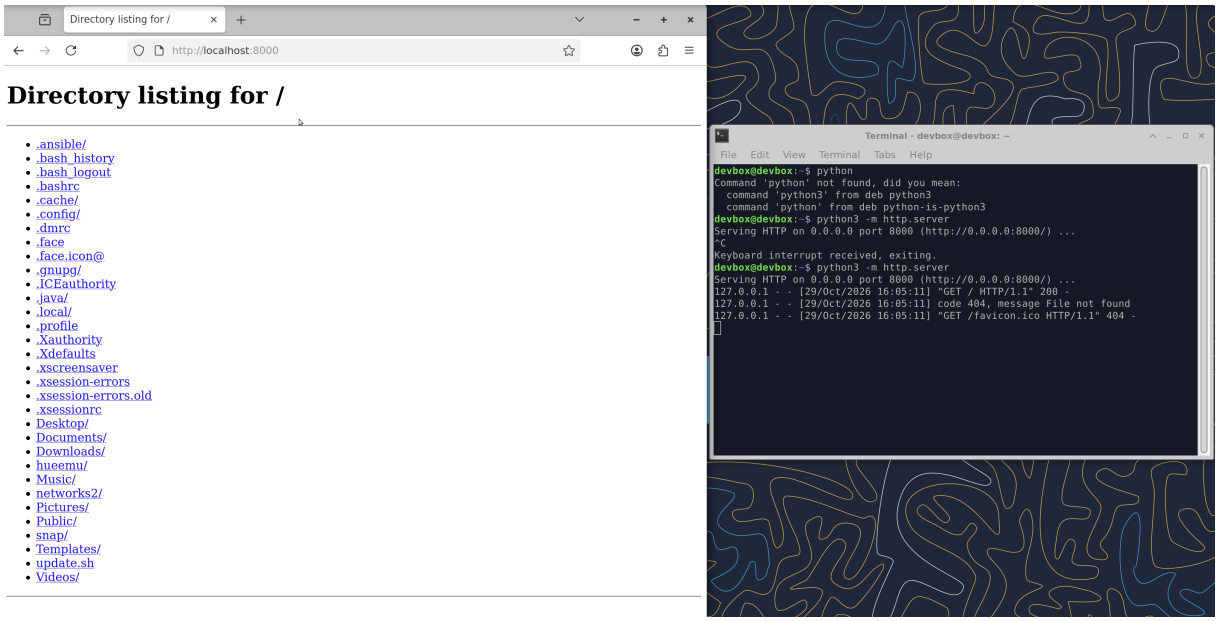
```
❯ curl --compressed "https://cantamus.app/packages/verovio-3.3.0/verovio.js" -H "If-None-Match: f1ee2cc0ca9c51
f2d05575f8c1436407-ssl-df" -I
HTTP/2 304
date: Tue, 18 May 2021 15:32:04 GMT
etag: "f1ee2cc0ca9c51f2d05575f8c1436407-ssl-df"
cache-control: public, max-age=0, must-revalidate
x-nf-request-id: 4aad8a5b-8368-4716-8ca2-9f9018a749b2
server: Netlify
vary: Accept-Encoding
```

3.2 3.2

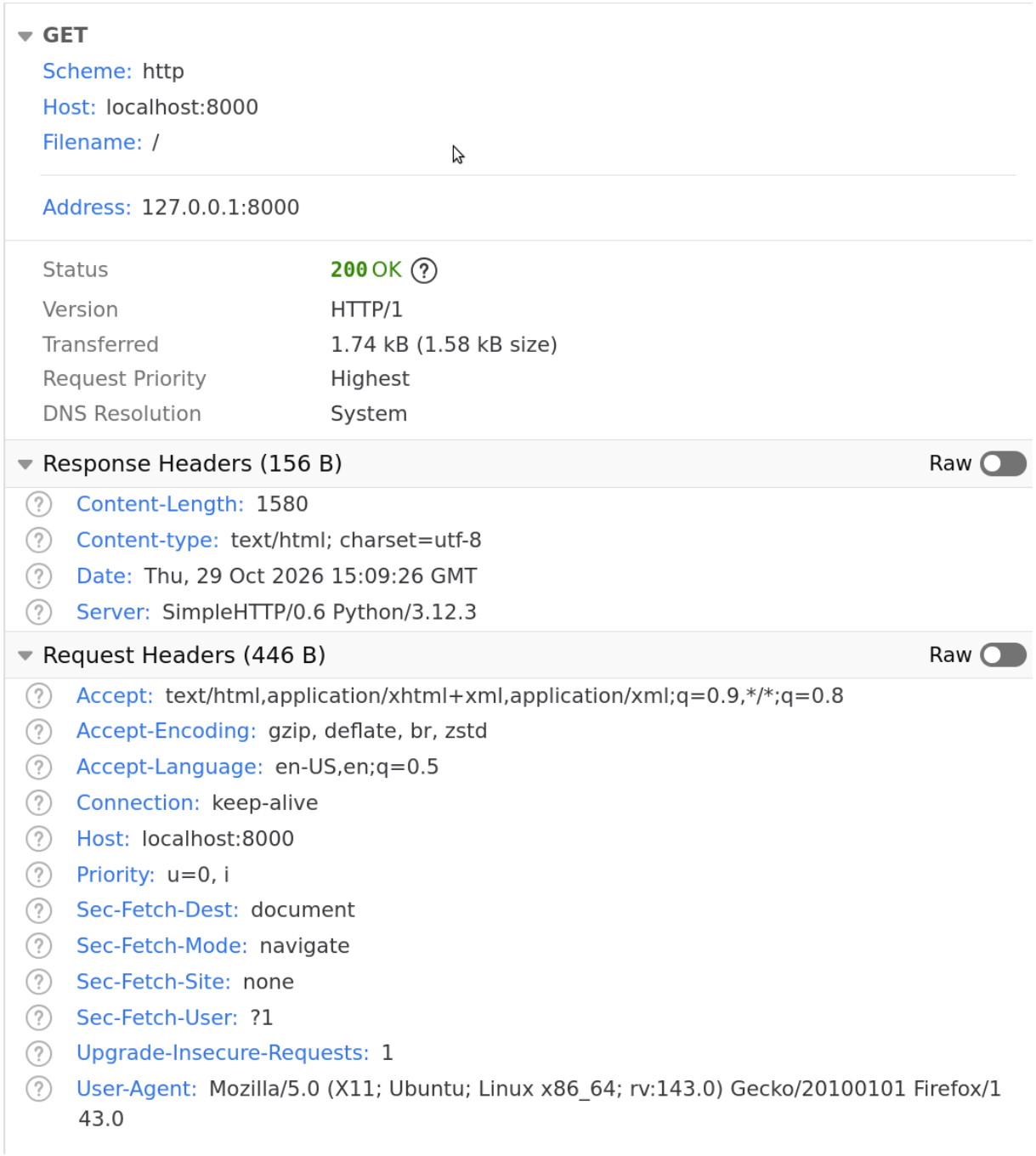
3.3 3.3

3.3.1 a

Es werden die Datein aus dem Ordner angezeigt, indem der Befehl ausgeführt wurde, hier das Homeverzeichnis.


Directory listing for /
http://localhost:8000
Directory listing for /
.ansible/
.bash_history
.bash_logout
.bashrc
.cache/
.config/
.dmrc
.face
.face.icon@
.gnupg/
.ICEauthority
.java/
.local/
.profile
.Xauthority
.Xdefaults
.xscreensaver
.xsession-errors
.xsession-errors.old
.xsessionrc
Desktop/
Documents/
Downloads/
hueemu/
Music/
networks2/
Pictures/
Public/
snap/
Templates/
update.sh
Videos/
Terminal - devbox@devbox: ~
File Edit View Terminal Tabs Help
devbox@devbox:~$ python
Command 'python' not found, did you mean:
command 'python3' from deb python3
command 'python' from deb python-is-python3
devbox@devbox:~$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
^C
Keyboard interrupt received, exiting.
devbox@devbox:~$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [29/Oct/2026 16:05:11] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Oct/2026 16:05:11] code 404, message File not found
127.0.0.1 - - [29/Oct/2026 16:05:11] "GET /favicon.ico HTTP/1.1" 404 -

3.3.2 b



3.3.3 c

Der Vollständige Code ist in OwnClient_v1_0.java zu finden.

3.3.4 d

Ab HTTP 1.1 wird der Header Ḧostöffiziell als verpflichtend gekennzeichnet, in v1.0 war dieser schon möglich, jedoch nicht verlangt. Viele Services erwarten diesen Header jedoch unabhängig vom Protokoll, da der Header dem Anbieter ermöglicht Anfragen durch Proxies und Loadbalancers zu routen ohne, dass das Ziel verloren geht. Außerdem ermöglicht dieser Header dem Webserver unterschiedliche Websites zu liefern ohne benötigt manuelle navigation.

3.3.5 e

Der Vollständige Code ist in OwnClient_v1_1.java zu finden.